

Lab 4

Operandi Allocati in Memoria e Array

Esempio: Load & Store words (usando etichette)

```
Messages Run I/O
Assemble: assembling /home/idrago/academic/teaching/2022/arch_i/labs/lab2/1_sum_numbers.asm
Assemble: operation completed successfully.
Assemble: assembling /home/idrago/academic/teaching/2022/arch_i/labs/lab4/3.a_sum_mem.asm
Assemble: operation completed successfully.
Error in /home/idrago/academic/teaching/2022/arch_i/labs/lab4/3.a_sum_mem.asm line 12: Runtime exception at 0x00400008: Load address not aligned to word boundary 0x100100
Step: execution terminated with errors.
```

```
.text
_start:
    la    t0, v1
    lw    t1, 1(t0)
    lw    t2, 4(t0)
    add   t3, t1, t2
    sw    t3, 8(t0)
```

Address + offset → lw → word aligned

Vincolo di allineamento: requisito per cui, in molte architetture, le parole devono iniziare sempre a indirizzi multipli di 4 e le parole doppie a indirizzi multipli di 8.

Approfondimento. In molte architetture, le parole devono iniziare sempre a indirizzi multipli di 4 e le parole doppie a indirizzi multipli di 8. Questo requisito si chiama **vincolo di allineamento** ed è comune a molte altre architetture (nel Capitolo 4 viene suggerito come l'allineamento produca una velocità maggiore nel trasferimento dei dati). I RISC-V e gli Intel x86 non hanno il vincolo di allineamento, mentre i MIPS sì.

Esercizio 1 - Media interi

Si scriva un programma in linguaggio RISC-V che carichi 4 numeri interi presente nella memoria in word contigue e calcoli il valore intero della loro media aritmetica (arrotondamento per difetto). Il valore calcolato va salvato in un'ulteriore posizione della memoria contigua a quelle usate per il calcolo.

- **In questo esercizio, utilizzare soltanto il set delle istruzioni "intere di base rv32i".**

Lab 5 - Esercizio 1 - Media interi

```
.globl _start
.data
    v1: .word 1
    v2: .word 2
    v3: .word 3
    v4: .word 4
    v5: .word 0
.text
_start:
    la    t1, v1          # get the v1 address
    lw    t2, 0(t1)        # copy the first word to t2
    lw    t3, 4(t1)        # copy the second word to t3
    add   t4, t2, t3        # sum the words and save it on register t4
    lw    t3, 8(t1)        # copy the third word to t3
    add   t4, t4, t3        # sum it and save on register t4
    lw    t3, 12(t1)       # copy the fourth word to t3
    add   t4, t4, t3        # sum it and save on register t4
    srai  t4, t4, 2         # shift to the right by 2 positions
    sw    t4, 16(t1)       # store the sum in the 5th word
```

Esercizio 2 - Max in memoria

Si scriva un programma in linguaggio RISC-V che calcoli il massimo tra tre numeri interi presenti nella memoria in word contigue (**variabili x, y e z**). Il valore massimo va salvato nella variabile **result**.

Esercizio 3 - Somma Array

Data una word in memoria (**variabile size**), seguita da "**size**" numeri interi in **word** contigue (array di **word**), scrivere un programma RISC-V che salva la somma degli elementi dell'array in una variabile identificata con l'etichetta **result**.

Esercizio 4 - Max Array

Data una word in memoria (**size**), seguita da "**size**" numeri interi in **word** contigue (array di **word**), scrivere un programma RISC-V che salva il maggiore valore presente nell'array in una variabile identificata con l'etichetta **result**.

Assumere che la dimensione dell'array sia sempre maggiore di 0.

Esercizio 5 - Numero di Dispari

Data una word in memoria (**size**), seguita da "**size**" **byte** (array di **byte**), scrivere un programma RISC-V che conta il numero totale di dispari tra gli elementi dell'array. Il valore ottenuto va salvato in una variabile identificata con l'etichetta "**result**".

Esercizio 6 - sorted

Scrivere il codice RISC-V che verifichi se un **array** di "**size**" elementi in memoria (**word** contigue) contiene una sequenza ordinata di numeri interi (crescente).

Il risultato va lasciato nella variabile result.

```
result = 1;
for (int i = 0; i < size-1; i++) {
    if (array[i] > array[i+1]) {
        result = 0;
        break;
    }
}
```

Esercizio 7 - Invert

Scrivere il codice per invertire un **array di word** in memoria (**size** elementi).

```
for (int i = 0; i < size/2; i++) {  
    int temp = array[i];  
    array[i] = array[size-i-1];  
    array[size-i-1] = temp;  
}
```

Esercizio 8 - Bubble sort

Scrivere il bubble sort in RISC-V per ordinare un **array di word** in memoria (**size** elementi).

```
int change = 1;
while (change) {
    change = 0;
    for (int i = 0; i < size; i++) {
        if (array[i] > array[i+1]) {
            int temp = array[i];
            array[i] = array[i+1];
            array[i+1] = temp;
            change = 1;
        }
    }
}
```